

GTSpase: A Geometric-Template-Driven Sparse Convolution Runtime for GPUs

Anonymous Author(s)

Abstract

Sparse convolution is the dominant computational primitive in 3D point-cloud perception. Its execution difficulty lies in that each output voxel’s set of active offsets (those pointing to non-empty input neighbors) is irregular and determined only at runtime. Existing work sit at opposite ends of a fundamental tension: some pursue no wasted computation by gathering each offset’s active output-offset pairs into an independent dense multiplication, but this fragments each output’s computation across many separate operations, sacrificing throughput; others maximize throughput by fusing all offsets of each output voxel into a single unified dense matrix multiplication, but inevitably waste arithmetic on inactive offsets.

We present GTSpase, a sparse convolution runtime that resolves this tension through *geometric templates*. We observe that the active-neighbor patterns of output voxels, although individually irregular, concentrate on a small number of recurring structures. GTSpase defines a compact set of geometric templates (fixed offset subsets), assigns each output voxel exclusively to its narrowest compatible template, and executes each template group as a narrower implicit GEMM. Within each template, all covered offsets are fused into a single unified dense GEMM (preserving high throughput); across templates, narrower widths eliminate wasted work on empty offsets (approaching no wasted computation). Consequently, GTSpase achieves **1.4–4.4×** end-to-end inference speedup over state-of-the-art sparse convolution engines on an NVIDIA RTX 4090 across representative 3D detection and segmentation workloads, while producing numerically identical results.

1 Introduction

Sparse convolution has become a key computational primitive in 3D point cloud perception, playing a central role in autonomous driving [9, 11, 15], robotics [13, 14, 24], and AR/VR [5, 8, 19]. Its importance stems from the intrinsic spatial sparsity of these domains: input features reside on 3D voxel grids in which only a tiny fraction of voxels are active. In widely used benchmarks, the active ratio (the fraction of occupied voxels) is extremely low: around 0.02% on nuScenes [2], about 0.04% on KITTI [7], and roughly 0.05% on SemanticKITTI [1]. Directly applying dense convolution would therefore waste substantial computation and memory on empty regions. The central difficulty is thus not the convolution operation itself, but how to efficiently organize and

execute the small number of irregularly distributed, input-dependent computations on GPUs.

Consider a $3\times 3\times 3$ sparse convolution kernel: each output voxel is associated with 27 spatial offsets, each pointing to a potential input neighbor. Under low-density inputs, most of these neighbors are empty; computation is needed only where a neighbor actually exists. We call such an (output, offset) pair an *active pair*. Unlike sparse matrix multiplication where the nonzero pattern is static, the set of active pairs here is determined entirely by the runtime distribution of input voxels and changes every frame. Each offset carries its own dense weight matrix shared across all its active pairs, so the operator repeatedly applies offset-specific weights over a dynamically generated, irregular set of spatial pairs. The number and distribution of active pairs cannot be known ahead of time, making efficient GPU scheduling fundamentally difficult.

Existing GPU sparse convolution engines sit at opposite ends of a fundamental tension between *no wasted computation* and *high throughput*. On one end, per-offset engines such as MinkowskiEngine [4] pursue no wasted computation: they collect active pairs separately for each offset and execute an independent multiplication per offset, so the total arithmetic is proportional to the number of genuinely active pairs. The consequence is that a single output voxel’s full computation is scattered across up to 27 separate operations that must be combined externally, preventing a cohesive, GPU-friendly execution and sacrificing throughput.

On the other end, fused-offset engines such as SpConv v2 [6] and TorchSparse++ [18] maximize throughput by fusing all 27 offsets into a single unified dense matrix multiplication (commonly called implicit GEMM), so that all contributions to each output voxel are produced within one cohesive operation. The consequence is that every offset position is computed uniformly, so a large fraction of arithmetic is spent on empty neighbors. Per-offset engines waste no computation but achieve only 19.5% of the throughput of fused-offset engines in our evaluation; fused-offset engines achieve high throughput but waste 50.7% of their computation on empty neighbors. Neither converts the GPU’s raw capability into effective throughput on genuinely active pairs.

Our key observation is that the active-neighbor patterns of output voxels, although individually irregular, concentrate on a small number of recurring structures, making it unnecessary to choose between fusing all 27 offsets and treating each independently. This concentration arises because active voxels in real scenes follow physical regularities (surfaces, edges, and clusters) that limit which offset combinations

actually co-occur. We can therefore define a small set of *geometric templates*, each a fixed subset of offsets, and assign every output voxel exclusively to the narrowest template that covers all its active neighbors. Output voxels sharing the same template then execute together as a single unified dense matrix multiplication over only that template’s offsets. Within each template, all covered offsets are fused into one cohesive operation (preserving high throughput); across templates, narrower widths than the full 27 offsets eliminate wasted work on empty neighbors (approaching no wasted computation).

Realizing this idea requires addressing two challenges. First, *template set design*: the template set must be compact (few templates, to limit execution branches) yet achieve tight coverage (low average assigned width across output voxels). Second, *heterogeneous execution*: output voxels assigned to different templates follow different-width compute paths on the same GPU; scheduling this intra-template homogeneous, inter-template heterogeneous workload efficiently is non-trivial.

We propose GTSparse, a sparse convolution runtime driven by geometric templates. At a high level, GTSparse constructs a compact template set offline from the kernel’s geometric structure, then at runtime assigns each output voxel to its tightest-fitting template, groups voxels by template, and executes each group as a narrower implicit GEMM over only the offsets specified by that template.

For template set design, GTSparse exploits the periodic structure of the kernel offsets to define candidate template families and uses data-driven greedy selection to assemble a compact set. Because the selected templates model the recurring geometric structures that arise from stable physical regularities across scenes, they are fixed and input-independent, constructed once offline and reused across all inputs without modification or runtime cost. In our evaluation across three workloads, 68.5–83.3% of output voxels are assigned to narrower templates, reducing the average number of offsets processed per voxel by 34.8–38.1% compared to full-width fusion. A full-width fallback template handles the remaining voxels whose patterns do not fit any narrower template.

For heterogeneous execution, GTSparse arranges all output voxels into a single contiguous stream, segmented by template and padded so that each tile (the smallest batch of output voxels processed together by GPU) contains only voxels of the same template. A single kernel launch processes the entire stream: each tile executes a dense matrix multiplication over only its template’s offsets, with no cross-template coordination needed. The result is intra-tile homogeneous dense execution at varying widths across tiles, achieved without multiple dispatches or explicit inter-template scheduling.

We evaluate GTSparse in both FP16 and FP32 precision with various GPU platforms on three representative model / dataset combinations that span across detection and segmentation tasks: SECOND [21] on KITTI [7], VoxelNeXt [3]

on nuScenes [2], and MinkUNet42 [4] on SemanticKITTI [1]. GTSparse achieves **1.3–3.6×** inference speedup over representative fused-offset engines (TorchSparse++ [18] and SpConv v2 [6]) and **1.9–6.3×** over the representative per-offset engine (MinkowskiEngine [4]) on desktop and datacenter GPUs, while producing numerically identical results. On the embedded Jetson AGX Orin platform, GTSparse delivers **1.2–2.3×** over fused-offset engines and **2.9–6.8×** over the per-offset engine.

The main contribution of this paper is the geometric templates as an execution abstraction for sparse convolution that resolves the tension between no wasted computation and high throughput. We present a template set construction method that combines a geometry-derived candidate space with data-driven greedy selection, producing a fixed, input-independent, and interpretable geometric templates. On top of this abstraction, we build a geometric-template-driven execution runtime that schedules intra-template homogeneous and inter-template heterogeneous computation within a single GPU kernel. We systematically evaluate GTSparse on real point-cloud models and datasets, demonstrating consistent speedups. By reducing sparse convolution latency, GTSparse extends the decision horizon and responsiveness of 3D perception systems in latency-critical applications such as autonomous driving, robotics, and AR/VR. Our implementation and reproduction scripts are publicly available at <https://github.com/eurosys-27-1183/GTSparse>.

2 Background and Motivation

This section formalizes the sparse convolution operator (§2.1), surveys the two dominant execution strategies and their limitations (§2.2), positions GTSparse relative to orthogonal optimization directions (§2.3), and quantifies the gap that motivates our design (§2.4).

2.1 Sparse Convolution Primer

Working scenario. 3D point-cloud perception models for autonomous driving [9, 11, 15], robotics [13, 14, 24], and AR/VR [5, 8, 19] typically voxelize raw sensor data (e.g., a LiDAR scan) onto a 3D grid and process it through a backbone composed of multiple sparse convolution layers. This computational demand is growing: next-generation LiDAR sensors produce increasingly dense point clouds (e.g., dual 128-beam sensors in Argoverse 2 [20]), and temporal fusion techniques that aggregate multiple sweeps into a single frame [22, 23] further multiply the number of active voxels per inference call. In these latency-critical deployments, sparse convolution directly determines the system’s reaction time: an autonomous vehicle must complete perception within the sensor frame interval (typically 50–100 ms at 10–20 Hz LiDAR) to maintain real-time responsiveness [9, 12], and any reduction in sparse convolution latency extends the time budget available for downstream planning and control.

Computation model. Because each input frame produces a different set of occupied voxels, which voxels participate in each convolution layer is unknown until runtime. We formalize this as follows. Let $\mathcal{A}_{\text{in}}$ and $\mathcal{A}_{\text{out}}$ denote the sets of active input and output voxels on the discretized 3D grid. A sparse convolution with a $k \times k \times k$ kernel defines $K = k^3$ spatial offsets $\mathcal{D} = \{d_1, \dots, d_K\}$. The output active set is determined by the input active set and the kernel geometry:

$$\mathcal{A}_{\text{out}} = \{p \mid \exists d \in \mathcal{D}, p + d \in \mathcal{A}_{\text{in}}\}. \quad (1)$$

For each output voxel $p \in \mathcal{A}_{\text{out}}$, let $\mathcal{N}(p) = \{d \in \mathcal{D} \mid p + d \in \mathcal{A}_{\text{in}}\}$ denote its set of valid offsets. The sparse convolution computes

$$y[p] = \sum_{d \in \mathcal{N}(p)} \mathbf{W}[d] \cdot \mathbf{x}[p+d], \quad (2)$$

where $\mathbf{W}[d] \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}$ is the weight matrix associated with offset d , $\mathbf{x}[p+d] \in \mathbb{R}^{C_{\text{in}}}$ is the input feature at voxel $p+d$, and $y[p] \in \mathbb{R}^{C_{\text{out}}}$ is the output feature at voxel p .

Submanifold vs. regular sparse convolution. Submanifold (SubM) sparse convolution constrains $\mathcal{A}_{\text{out}} = \mathcal{A}_{\text{in}}$, preserving the spatial sparsity pattern across layers. Consequently, for consecutive SubM layers with the same kernel size, the per-offset mapping from each output voxel to its valid input neighbors remains identical and need only be constructed once. In representative 3D detection backbones such as SECOND [21] and VoxelNeXt [3], SubM layers account for the overwhelming majority of convolution layers. All existing engines [4, 6, 18], including GTSparse, exploit this invariance by reusing runtime metadata across SubM layers. Regular (full) sparse convolution applies stride-2 down-sampling, producing a new output active set each layer. Its runtime metadata must be rebuilt per layer.

2.2 Existing Execution Strategies

Per-offset engines. Per-offset engines such as MinkowskiEngine [4] partition computation by kernel offset: for each offset d , the engine gathers active pairs into a contiguous buffer, invokes a dense GEMM, and scatters results to the output. This wastes no computation (empty offsets are skipped), but fragments the workload into up to K unevenly sized GEMMs. At low active ratios many offsets produce only a handful of pairs, yielding thin GEMMs that cannot saturate the GPU. Each output voxel’s result must be accumulated externally across up to K scatter operations. This external accumulation is prone to precision loss under FP16, and the thin irregular GEMMs cannot meet Tensor Core alignment requirements, confining MinkowskiEngine to FP32 CUDA cores. Subsequent systems (TorchSparse [16] adaptive grouping, SpConv v1 fused dispatch) mitigate launch overhead, but per-offset fragmentation remains the root cause of low raw throughput.

Fused-offset engines. Fused-offset engines fuse all K offsets into a single dense matrix multiplication (an approach known as implicit GEMM), so that each output voxel’s complete result is produced within one unified operation. SpConv v2 [6] pioneered this approach, introducing bitmask-based sorting to reorder output voxels so that those with similar active-neighbor patterns are grouped together, enabling warp-level skipping of contiguously empty offset positions. TorchSparse++ [18] later showed that with sufficiently optimized GEMM tile configurations, the full implicit GEMM without explicit skipping can match or exceed the performance of sort-and-skip, because the overhead of sorting and the limited skip granularity often offset the savings from skipping. In both systems, the execution is a unified dense GEMM that is GPU-friendly and Tensor Core compatible, but the fused offset dimension inevitably includes empty neighbor positions, so a fraction of arithmetic is spent on empty neighbors that contributes nothing to the final result.

2.3 Orthogonal Optimization Directions

Several lines of work improve sparse convolution performance through mechanisms orthogonal to the execution strategy itself. *Compiler-based* optimizations (TorchSparse++ TVM codegen [18], PCEngine [17]) apply operator fusion and hardware-specific code generation within an existing execution model. *Profiling-based autotuning* (e.g., PCEngine [17] switching between gather-scatter and implicit GEMM based on layer density, TorchSparse++ [18] autotuning tile configurations) adapts execution parameters but does not change the underlying execution model. *Hardware acceleration* approaches (e.g., PointAcc [10]) change the compute substrate but not the software-level execution model. These directions are orthogonal to GTSparse: GTSparse proposes a new execution model, and the above optimizations remain applicable on top of it.

2.4 Motivation: The Effective Throughput Gap

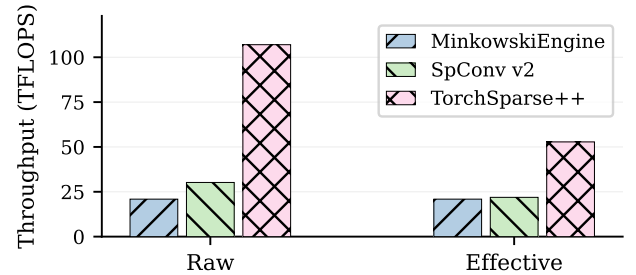


Figure 1. Raw vs. effective computation throughput for existing sparse convolution engines (VoxelNeXt/nuScenes, sw=1, RTX 4090). SpConv v2 and TorchSparse++ run in FP16; MinkowskiEngine runs in FP32 as it does not support FP16.

In the target inference scenario (batch size 1, single frame per forward pass), sparse convolution layers dominate end-to-end latency. The total time spent on each layer includes both preprocessing (building the neighbor map or runtime metadata) and the convolution kernel itself. Reducing either component directly lowers end-to-end inference time. To understand where existing systems lose efficiency, we distinguish two throughput metrics:

- *Raw computation throughput*: total FLOPs issued by the hardware divided by wall-clock time, regardless of whether the computation is useful. This is what standard GPU profilers report.
- *Effective computation throughput*: only the FLOPs corresponding to genuinely active pairs, divided by wall-clock time.

We formalize effective throughput as follows. For a single sparse convolution layer l with $N_{\text{pairs}}^{(l)}$ active pairs and channel widths $C_{\text{in}}^{(l)}, C_{\text{out}}^{(l)}$, the useful work is $N_{\text{pairs}}^{(l)} \times C_{\text{in}}^{(l)} \times C_{\text{out}}^{(l)} \times 2$ FLOPs (the factor of 2 accounts for multiply and accumulate). For a model with L sparse convolution layers executed in total wall-clock time t_{wall} (including both preprocessing and kernel execution for all layers), the aggregate effective throughput is

$$T_{\text{eff}} = \frac{\sum_{l=1}^L N_{\text{pairs}}^{(l)} \times C_{\text{in}}^{(l)} \times C_{\text{out}}^{(l)} \times 2}{t_{\text{wall}}}. \quad (3)$$

A system that wastes arithmetic on empty neighbors will show raw throughput much higher than effective throughput; a system that spends excessive time on preprocessing will show low effective throughput even if its kernel is efficient. Only when both proportionality and speed are high does T_{eff} approach the GPU’s peak.

Figure 1 illustrates this gap. MinkowskiEngine performs almost exclusively useful arithmetic (raw $\approx$ effective at 20.9 TFLOPS), but both values are low because per-offset fragmentation prevents the GPU from reaching high throughput. TorchSparse++ achieves the highest raw throughput (107.0 TFLOPS) by executing a single fused GEMM over all offsets, but only 49.3% of that computation corresponds to active pairs (52.8 TFLOPS effective); the rest is wasted on empty neighbors. SpConv v2 reduces wasted computation through sort-and-skip (72.5% proportionality), but its preprocessing overhead (sorting, hashtable construction) limits raw throughput to only 30.2 TFLOPS, yielding 21.9 TFLOPS effective. No existing system achieves both high raw throughput and high proportionality simultaneously, leaving substantial room for improvement.

3 Overview

Figure 2 illustrates the GTSparse pipeline alongside the two existing execution strategies. GTSparse targets single-frame inference: each input frame’s active voxel set is independently classified and executed with no cross-frame state or

dependency. Given a set of active input voxels (Figure 2a), fused-offset engines (Figure 2b) achieve high throughput by fusing all offsets into a single dense GEMM, but uniformly compute the full K -wide reduction for every output voxel, wasting arithmetic on empty positions. Per-offset engines (Figure 2c) eliminate wasted computation by processing each offset independently, but scatter a single output voxel’s computation across up to K separate operations, preventing cohesive GPU execution. GTSparse (Figure 2d) resolves this tension through a three-step pipeline.

Step 1: Template classification. For each output voxel, the runtime builder probes which neighbors are present, producing a bitmask of active offsets. It then assigns the voxel to the narrowest geometric template whose offset subset covers this bitmask (by comparing the template’s reject bitmask). A full-width fallback template guarantees universal coverage. Each voxel is classified independently in parallel on the GPU, requiring no global sorting or device-to-host synchronization.

Step 2: Layout construction. Output voxels sharing the same template are grouped and arranged into a single contiguous tile stream. Each group is padded to tile boundaries so that every tile contains only voxels of one template. This layout enables the kernel to determine its compute path by reading the template identity once per tile.

Step 3: Geometric-Template-driven Kernel execution. A single GPU kernel launch processes the entire tile stream. Each tile executes a dense matrix multiplication over only its template’s offsets: tiles assigned to narrower templates iterate over fewer offsets, while the full-width fallback iterates over all K . Within each tile, the computation is a uniform dense GEMM (preserving high throughput); across tiles, narrower widths eliminate arithmetic on empty neighbors (approaching no wasted computation). No cross-template coordination or multiple dispatches are needed.

Together, these three steps give GTSparse a lightweight and complete execution path that resolves the tension between no wasted computation and high throughput, directly improving effective computation throughput. The template set is constructed offline and reused across all inputs without modification; how to select a compact set that achieves tight coverage will be discussed in the next section.

4 Design

This section details the design of GTSparse. We first describe the offline template set construction (§4.1), which produces the fixed geometric template set used across all inputs. We then describe the runtime components that correspond to the pipeline in §3: the runtime builder that performs template classification and layout construction (§4.2), and the kernel execution model that processes the resulting tile stream (§4.3).

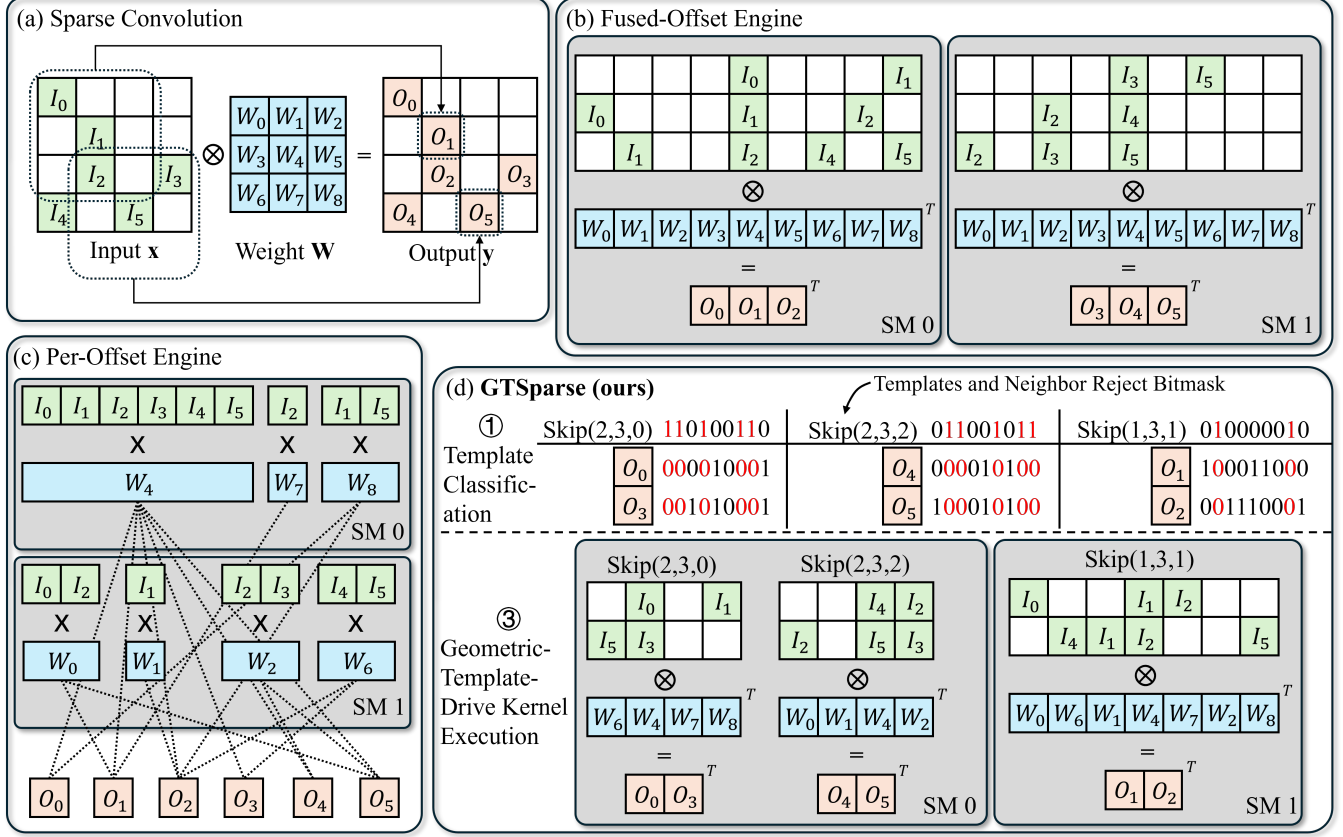


Figure 2. GTSparse overview, illustrated on a 2D 3×3 submanifold sparse convolution with 5 active input/output voxels. The GPU has two streaming multiprocessors (SMs), each running one thread block. (a) Sparse convolution setup: input/output active voxels and the 3×3 kernel weights. (b) Implicit-GEMM engines fuse all 9 offsets into one GEMM; white cells denote wasted computation on empty neighbor positions. (c) Per-offset engines issue a separate GEMM per offset; each output voxel’s computation is scattered across up to K operations that must be accumulated externally (dashed lines). (d) GTSparse classifies each output voxel to its narrowest template (e.g., Skip(2,3,0), further elaborated in §4.1) by comparing its active neighbor pattern bitmask (in column-major order) with the template neighbor reject bitmask (Step 1) and executes each tile over only that template’s offsets (Step 3), reducing wasted computation while preserving dense execution. Step 2 (layout construction) is omitted for brevity.

4.1 Geometric Template Set Construction

The template set design challenge is to select a compact set of geometric templates such that, after assigning each output voxel to its narrowest compatible template, the average assigned width is significantly smaller than the full offset count. We frame this as a structured variant of set cover and solve it with a two-stage pipeline: geometry-driven candidate generation followed by data-driven greedy selection.

Problem formulation. Recall from §2.1 that a sparse convolution kernel defines K spatial offsets $\mathcal{D}$, and each output voxel $p \in \mathcal{A}_{\text{out}}$ has a set of valid offsets $\mathcal{N}(p) \subseteq \mathcal{D}$. A *template set* is a collection of offset subsets $\mathcal{T} = \{T_1, \dots, T_m\}$, $T_i \subseteq \mathcal{D}$. Each output voxel p is assigned to its narrowest compatible template, i.e., the smallest T_i that contains all of

p ’s active offsets:

$$a(p) = \arg \min_{T_i \in \mathcal{T}} |T_i| \quad \text{s.t.} \quad \mathcal{N}(p) \subseteq T_i. \quad (4)$$

The objective is to minimize the average assigned width:

$$\min_{\mathcal{T}} \frac{1}{|\mathcal{A}_{\text{out}}|} \sum_{p \in \mathcal{A}_{\text{out}}} |a(p)| \quad \text{s.t.} \quad \mathcal{T}_0 \subseteq \mathcal{T}, \quad (5)$$

where $\mathcal{T}_0$ is a fixed *base template set* (defined below) that must always be included.

Base templates. Any template set includes two base templates by default:

- **Center** (width 1): covers output voxels whose only active neighbor is the center offset. These voxels are common at very low densities.

- **Full-width fallback** (width K): covers all offsets. It works as a fallback template guaranteeing that every output voxel can be assigned regardless of its active pattern.

Candidate generation. Rather than searching the exponential space of all possible offset subsets (2^K possibilities), we exploit the geometric structure of the kernel to define a tractable candidate space. We first fix a canonical ordering of the K offsets into a sequence $\sigma = (\sigma_0, \sigma_1, \dots, \sigma_{K-1})$. On this sequence, we define three parametric types of candidate templates:

- **Contiguous-hole**(h, s). Retain all offsets except a contiguous segment of length h starting at position s :

$$T = \{\sigma_i \mid (i - s) \bmod K \geq h\} \cup \{\text{center}\}.$$

- **Periodic-skip**(n, q, s). Starting at position s , remove n out of every q consecutive offsets:

$$T = \{\sigma_i \mid (i - s) \bmod q \geq n\} \cup \{\text{center}\}.$$

- **Hybrid**(n, q, h, s). Apply both a periodic-skip pattern and a contiguous hole simultaneously.

In all three types, the center offset is unconditionally included regardless of the parametric pattern. Because the same template set is used for both submanifold and full convolution layers; under submanifold convolution ($\mathcal{A}_{\text{out}} = \mathcal{A}_{\text{in}}$), the center offset is always active by definition, so any template that excludes it cannot cover any output voxel.

For each combination of parameters other than s (e.g., h for contiguous-hole, or n, q for periodic-skip), we fix these parameters and collect all valid instantiations over s into a single *family* F , which serves as the atomic unit of selection. This grouping ensures that when a family is selected, all of its phases (all possible s) are available for assignment; otherwise, voxels whose active patterns align only with a specific phase would lose coverage. The candidate space C is then the set of all families obtained by enumerating all valid parameter combinations across the three family types (contiguous-hole, periodic-skip, hybrid). A voxel p is *covered* by a family if at least one of its variants T satisfies $\mathcal{N}(p) \subseteq T$. Since C is derived entirely from the kernel’s offset geometry, it does not depend on any particular input data.

Greedy selection. We solve the optimization problem in Eq. 5 approximately using a greedy algorithm over the candidate space C . The core idea is to iteratively add the family that maximally reduces the objective (average assigned width) at each step.

To quantify the gain of adding a candidate family F to the current template set $\mathcal{T}$, we define its *marginal offset reduction*. Let $a_{\mathcal{T} \cup F}(p)$ denote the assignment of voxel p under the augmented set $\mathcal{T} \cup F$. The marginal offset reduction is:

$$\Delta(F) = \sum_{p \in \mathcal{A}_{\text{out}}} (|a(p)| - |a_{\mathcal{T} \cup F}(p)|), \quad (6)$$

Algorithm 1 Greedy Template Set Selection

Require: Candidate space C , base set $\mathcal{T}_0$, representative output voxels $\mathcal{A}_{\text{out}}$, max rounds R , threshold ϵ

Ensure: Template set $\mathcal{T}$

```

1:  $\mathcal{T} \leftarrow \mathcal{T}_0$ 
2: Compute initial assignment  $a(p)$  for all  $p \in \mathcal{A}_{\text{out}}$ 
3: for round = 1, 2, ...,  $R$  do
4:   for each  $F \in C$  do
5:     Compute  $\Delta(F)$  via Eq. (6)
6:   end for
7:    $F^* \leftarrow \arg \max_F \Delta(F)$ 
8:   if  $\Delta(F^*) < \epsilon$  then
9:     break
10:  end if
11:   $\mathcal{T} \leftarrow \mathcal{T} \cup F^*$ ;  $C \leftarrow C \setminus \{F^*\}$ 
12:  Update  $a(p)$  for all  $p \in \mathcal{A}_{\text{out}}$ 
13: end for
14: return  $\mathcal{T}$ 

```

which measures the total number of offsets eliminated if F were added. Starting from the base set $\mathcal{T}_0$, the algorithm selects the family with the largest Δ , adds it to $\mathcal{T}$, updates all assignments, and repeats until Δ falls below a threshold ϵ .

We formalize this as Algorithm 1. Line 1 initializes the template set to the base set. Line 2 computes the initial voxel assignments under $\mathcal{T}_0$. The main loop (lines 3–10) evaluates all remaining candidates, selects the best one, and updates assignments. The algorithm terminates either when the maximum number of rounds R is reached or when the marginal gain falls below ϵ . Upon termination, the selected families together with $\mathcal{T}_0$ form the final template set $\mathcal{T}$ that approximately minimizes Eq. 5.

Resulting template set. We run Algorithm 1 with $\epsilon = 1$, using the first 16 frames of the training split of KITTI [7] with SECOND [21] as the representative sample (aggregating $\mathcal{A}_{\text{out}}$ across all SubM layers). As shown in Figure 3a, the algorithm converges in two rounds for 3x3x3 convolution kernels, selecting:

- Round 1: periodic-skip(1, 3), producing 3 variants of width 19.
- Round 2: periodic-skip(2, 3), producing 3 variants of width 10.

The widths (10 and 19) include the unconditionally added center offset. Together with the base set $\mathcal{T}_0$ (center of width 1, full-width fallback of width K), the final template set contains 8 templates.

Figure 3b shows the template assignment distribution. Approximately 70.3% of output voxels are assigned to templates narrower than the full-width fallback, reducing the average template width to 18.9 (vs. 27 for full-width fusion). Importantly, the template set is input-independent: constructed

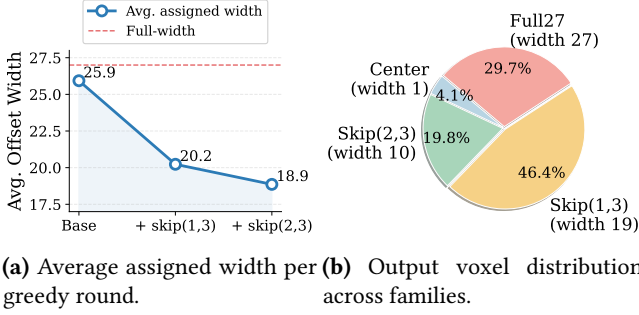


Figure 3. Greedy template set selection on KITTI+SECOND (16 frames). (a) Average assigned template width converges rapidly in two rounds. (b) Approximately 70% of output voxels are assigned to narrower templates (width < 27).

once offline from a representative sample, it achieves stable coverage across different datasets (KITTI, nuScenes, SemanticKITTI) without modification in our evaluation.

Why geometric templates achieve tight coverage. The effectiveness of a small template set rests on a physical regularity: real-world 3D scenes are composed predominantly of low-dimensional geometric structures whose local neighborhoods follow predictable offset patterns. Surfaces (ground planes, building facades) produce planar neighborhoods where offsets along the surface normal are largely unoccupied. Edges (road curbs, pole boundaries) produce linear neighborhoods with even fewer active offsets. Isolated voxels (distant objects, noise) activate only the center offset. These recurring structures concentrate the empirical distribution of active-neighbor patterns into a small number of modes, which the periodic-skip families naturally capture: skip(2, 3) (width 10) covers the sparse linear and isolated-surface patterns, while skip(1, 3) (width 19) covers the denser planar neighborhoods. Because these geometric primitives are universal across outdoor LiDAR scenes, the template set transfers across datasets without retuning.

Generality. Although the template set above is instantiated for a $3 \times 3 \times 3$ kernel, the construction pipeline generalizes to other kernel shapes. Smaller kernels (e.g., $3 \times 3 \times 1$) are handled by disabling the irrelevant offset dimensions, collapsing the template set to a subset. Larger kernels can be padded to a multiple of 3 and the same periodic candidate generation applied, since the underlying regularity stems from the spatial structure of the data (surfaces, edges, clusters) rather than the kernel size itself. More broadly, the entire pipeline (candidate generation + greedy selection) is domain-agnostic: applying it to a different application domain requires only re-running Algorithm 1 on representative data from that domain to discover the template set best suited to its spatial patterns.

4.2 Runtime Builder

Given the template set $\mathcal{T}$ produced offline, the runtime builder constructs, for each input frame, the execution metadata that the kernel consumes. It implements the template classification and layout construction steps described in §3.

Template classification. For each output voxel p , the builder first determines $\mathcal{N}(p)$ by probing which of the K neighbor positions are occupied. Representing $\mathcal{N}(p)$ as a bitmask $b(p) \in \{0, 1\}^K$, we precompute for each template $T_i \in \mathcal{T}$ a *reject mask* $r_i \in \{0, 1\}^K$ corresponding to $\mathcal{D} \setminus T_i$ (the offsets not in T_i). The assignment (Eq. 4) is then computed as:

$$a(p) = \arg \min_{T_i \in \mathcal{T}} |T_i| \quad \text{s.t.} \quad b(p) \text{ AND } r_i = 0, \quad (7)$$

which is equivalent to Eq. 4 but reduces the subset test $\mathcal{N}(p) \subseteq T_i$ to a single bitwise operation. Each voxel’s classification is independent, requiring $O(m)$ bitwise operations per voxel.

Tile stream layout. Let $\mathcal{P}_i = \{p \in \mathcal{A}_{\text{out}} : a(p) = T_i\}$ be the set of voxels assigned to template T_i , and let B_M be the tile size. The builder pads each group to $\hat{n}_i = \lceil |\mathcal{P}_i| / B_M \rceil \cdot B_M$ voxels (filling padding positions with invalid markers), then concatenates all groups into a contiguous tile stream of total length $N = \sum_i \hat{n}_i$. This layout guarantees that every tile of B_M consecutive positions contains only voxels of the same template. The kernel reads the template identity once per tile to determine its compute path, with no further scheduling decisions needed. Padding positions produce zero contributions during computation and are skipped during writeback.

Overhead and reuse. The builder executes entirely on the GPU with no device-to-host synchronization. Its time complexity is $O(|\mathcal{A}_{\text{out}}| \cdot (K + m))$: $O(K)$ per voxel to probe neighbors and $O(m)$ for classification. With our template set ($m = 8$, $K = 27$), $K > m$ and thus neighbor probing dominates. For submanifold convolution layers ($\mathcal{A}_{\text{out}} = \mathcal{A}_{\text{in}}$), the assignment $a(p)$ and the tile stream are invariant across layers sharing the same kernel size, so the metadata is built once and reused. For full convolution layers, the builder runs once per layer.

4.3 Kernel Execution Model

The kernel computes Eq. 2 for all output voxels by consuming the tile stream. Its execution adopts the same implicit-GEMM tiling strategy as existing fused-offset engines (SpConv v2, TorchSparse++), with one modification: the reduction dimension (number of offsets) is no longer fixed at K for all tiles, but equals $|a(p)|$ for each tile according to its assigned template.

Tiling structure. The workload is decomposed into tiles of size $B_M \times B_N$ (output voxels $\times$ output channels), identical to standard implicit GEMM. The inner loop reduces over offsets and input channels in steps of B_K . In standard

Algorithm 2 Template-Driven Kernel Execution Model

```
1: for each tile  $(b_m, b_n)$  in the tile stream in parallel do
2:    $T_i \leftarrow$  template of tile  $b_m$ 
3:   Initialize accumulator  $Y \leftarrow \mathbf{0}_{B_M \times B_N}$ 
4:   for  $j = 1, \dots, |T_i|$  do
5:     for  $k = 1, \dots, \lceil C_{\text{in}}/B_K \rceil$  do
6:        $X \leftarrow$  gather input block from offset  $d_j$ , channel
       tile  $k$ 
7:        $W \leftarrow$  load weight block for offset  $d_j$ , channel
       tile  $k$ 
8:        $Y \leftarrow Y + X \cdot W$ 
9:     end for
10:  end for
11:  Write  $Y$  to output (skip padding positions)
12: end for
```

implicit GEMM, this loop runs for $K \times \lceil C_{\text{in}}/B_K \rceil$ steps. In GTSparse, it runs for $|a(p)| \times \lceil C_{\text{in}}/B_K \rceil$ steps, where $|a(p)| \leq K$. Each step is identical: gather input features, load the corresponding weight slice, and perform a dense matrix-multiply-accumulate. The only difference is how many steps each tile executes.

Heterogeneous dispatch. A single kernel launch processes the entire tile stream, as summarized in Algorithm 2. All tiles execute in parallel (line 1). Each tile reads its template identity once (line 2), which determines the loop bound in line 4: tiles with narrow templates iterate over fewer offsets, while tiles with the full-width fallback iterate over all K . The inner loop (lines 4–9) performs a standard dense matrix-multiply-accumulate per offset per channel tile. No cross-tile coordination is needed. The execution is intra-template homogeneous (each tile runs a uniform dense GEMM, preserving high throughput) and inter-template approaching no wasted computation (narrower tiles skip offsets known to be empty), improving T_{eff} (Eq. 3).

5 Implementation

GTSparse is implemented as a PyTorch C++/CUDA extension that exposes a drop-in replacement for existing sparse convolution layers. The runtime builder and convolution kernel are both implemented in CUDA. The implementation consists of approximately 4,300 lines of CUDA kernel code and 1,400 lines of Python/C++ glue. The system supports both FP32 (SIMT) and FP16 (Tensor Core).

Kernel integration. The convolution kernel for the full-width fallback template integrates the implicit-GEMM kernel from the official code of TorchSparse++ [18]. Narrower templates are implemented by adapting the same kernel structure with a shorter inner loop (iterating over $|a(p)| < K$ offsets instead of K). We also adopt their dispatch logic for selecting tile configurations based on dtype and input/output

channel dimensions. These ensure that the performance characteristics of the full-width path match TorchSparse++ exactly, and any speedup observed on narrower templates is attributable solely to the reduced iteration count.

Worst-case pre-allocation. The builder appends each output voxel’s payload (its template-specific input row indices) to the buffer of its assigned template. Since the per-template counts $|\mathcal{P}_i|$ are unknown before classification completes, a naïve two-pass approach would first count, allocate, then scatter in a second pass. GTSparse instead pre-allocates all payload buffers at the worst-case size: each buffer is sized as if all $|\mathcal{A}_{\text{out}}|$ output voxels belonged to that template. Concretely, let $\hat{N} = \lceil |\mathcal{A}_{\text{out}}|/B_M \rceil \cdot B_M$ be the tile-padded output count. For each template $T_i \in \mathcal{T}$, the builder allocates a payload buffer of $\hat{N} \times |T_i|$ integers. This allows classification and payload writing to proceed in a single pass: each voxel appends its payload to its assigned template’s buffer immediately upon classification, without inter-voxel data dependency. The total pre-allocated metadata is $\hat{N} \times \sum_{i=1}^m |T_i|$ integers, which is bounded for a given $|\mathcal{A}_{\text{out}}|$.

Template compilation and implementation scope. The geometric template set is constructed offline using the greedy selection pipeline described in §4.1. The resulting reject masks and offset mappings are compiled as constants into the CUDA kernel, incurring no runtime overhead for template metadata lookup. The neighbor probing step in the builder uses a GPU hash table for coordinate lookup, shared with the SubM runtime when available.

6 Evaluation

6.1 Experimental Setup

Hardware platforms. We evaluate GTSparse on six GPU platforms spanning the deployment spectrum of 3D perception inference: an NVIDIA Jetson AGX Orin (embedded, Ampere architecture with unified memory), an RTX 2080 Ti (desktop Turing), an RTX 3080 (desktop Ampere), an RTX 4070 Laptop (mobile Ada Lovelace), an RTX 4090 (desktop Ada Lovelace), and an A100 (datacenter Ampere). This selection covers the hardware range from low power edge devices to datacenter GPUs used for cloud-based inference.

Workloads. We select three representative model/dataset combinations that span detection and segmentation tasks at different density levels, summarized in Table 1: SECOND [21] on KITTI [7], representing low-density single-sweep detection; VoxelNeXt [3] on nuScenes [2], representing moderate-density detection, evaluated under both single-sweep and 10-sweep configurations; and MinkUNet42 [4] on SemanticKITTI [1], representing higher-density semantic segmentation. Multi-sweep temporal fusion is a standard practice in LiDAR detection pipelines [2, 11] that aggregates consecutive frames into a single voxel grid via ego-motion compensation, increasing the density of the sparse convolution workload;

Table 1. Workload statistics. Active ratio is the fraction of occupied voxels in the spatial grid, averaged over the evaluation split. sw denotes the number of LiDAR sweeps fused per frame.

Model (Params)	Dataset	Voxels	Frames	Active Ratio	
				sw=1	sw=10
SECOND (5.3M)	KITTI	92M	7,518	0.04%	—
VoxelNeXt (7.4M)	nuScenes	85M	6,008	0.02%	0.11%
MinkUNet42 (39.2M)	Sem.KITTI	138M	4,071	0.05%	—

nuScenes natively provides the multi-sweep annotations required for this evaluation.

Baselines. We compare against three state-of-the-art sparse convolution engines, each obtained from its official GitHub repository at the latest available version and run with default configurations: MinkowskiEngine [4] (per-offset gather-GEMM-scatter, referred to as *Minkowski*), SpConv v2 [6] (fused-offset with sorted bitmask skipping, referred to as *SpConv*), and TorchSparse++ [18] (fused-offset without skipping, referred to as *TS++*). All systems run under PyTorch 2.1, CUDA 12.1, and cuDNN 8.9. Minkowski only supports FP32: its per-offset execution model accumulates each output voxel’s result across up to K separate GEMM invocations, requiring FP32 accumulation to preserve numerical accuracy. In contrast, fused-offset engines (including GTSparse) produce each output within a single fused operation whose internal FP32 accumulator is sufficient for FP16 inputs, enabling native FP16 execution without precision loss.

Metrics. Following the convention of TS++ [18], for detection workloads we measure the total time spent in sparse convolution layers (excluding model-specific non-convolution detection heads); for the segmentation workload we measure the full inference pipeline. We evaluate under both FP16 (Tensor Core) and FP32 (SIMT). All systems produce numerically identical outputs under the same precision setting.

6.2 End-to-End Performance

Figure 4 reports inference latency across all GPU platforms and workloads under both FP16 (Tensor Core) and FP32 (SIMT) precision. GTSparse outperforms every baseline in all configurations. Since Minkowski only supports FP32, the speedup ranges below are reported against the two fused-offset baselines (SpConv and TS++) that share the same precision as GTSparse in each configuration. Under FP16, GTSparse achieves 1.11–3.60 $\times$ speedup across five GPU platforms; under FP32 on RTX 2080 Ti, the speedup is 1.26–1.73 $\times$. All systems produce numerically identical outputs under the same precision setting.

GPU architecture trend. The speedup correlates with GPU compute throughput. On the highest-throughput FP16

platforms (RTX 4090 and A100), GTSparse achieves 1.40–3.60 $\times$ over fused-offset baselines: Tensor Core arithmetic is fast enough that wasted cycles on empty offsets dominate execution time, and template-driven narrowing eliminates precisely these cycles. Mid-range GPUs (RTX 3080, RTX 4070 Laptop) show 1.27–3.02 $\times$ speedup with the same trend at reduced magnitude. On AGX Orin, where both compute throughput and memory bandwidth are constrained, the combined speedup is the smallest across FP16 platforms (1.11–2.26 $\times$): execution is more readily limited by memory latency than by compute, reducing the marginal benefit of eliminating wasted arithmetic. On RTX 2080 Ti under FP32 (SIMT-only path), GTSparse achieves 1.27–1.62 $\times$ over TS++, 1.26–1.73 $\times$ over SpConv, and 1.85–2.28 $\times$ over Minkowski, confirming that template-driven execution benefits the SIMT datapath equally by reducing issued arithmetic instructions rather than relying on Tensor Core utilization.

Workload density trend. The density effect is most clearly isolated by comparing VoxelNeXt under single-sweep (active ratio 0.02%) versus 10-sweep (0.11%), where only input density changes. As density increases, the speedup over TS++ narrows consistently: on RTX 4090 from 1.47 $\times$ to 1.40 $\times$, on A100 from 1.42 $\times$ to 1.27 $\times$, and on RTX 3080 from 1.40 $\times$ to 1.30 $\times$. Denser inputs fill more offsets per output row, shifting classification toward wider templates and reducing the width advantage. SECOND achieves the highest speedups on high-throughput GPUs (1.64 $\times$ on RTX 4090) because its smaller channel widths make each layer more compute-bound, amplifying the benefit of eliminating wasted arithmetic.

Baseline behavior. Minkowski operates exclusively in FP32 because its per-offset execution model accumulates each output across up to K separate GEMM invocations, requiring an FP32 external accumulator to preserve numerical accuracy. Without Tensor Core acceleration, it is the slowest system in most cases; under FP32 on RTX 2080 Ti it remains the slowest due to the inherent fragmentation of per-offset dispatch. SpConv’s sorted implicit GEMM incurs sorting overhead that outweighs its computation reduction on high-throughput GPUs with sparse detection workloads, making it slower than TS++ in most FP16 configurations. However, on AGX Orin where throughput and bandwidth are both limited, sorting’s computation reduction outweighs its cost (e.g., MinkUNet: SpConv 159 ms vs. TS++ 254 ms). Under FP32 on RTX 2080 Ti, SpConv similarly benefits from sorting since SIMT throughput is lower. TS++ avoids sorting overhead via unsorted implicit GEMM and is the strongest baseline on desktop and datacenter FP16 platforms, but it uniformly executes the full K -offset reduction for every output row regardless of actual neighbor occupancy, incurring wasted computation proportional to sparsity.

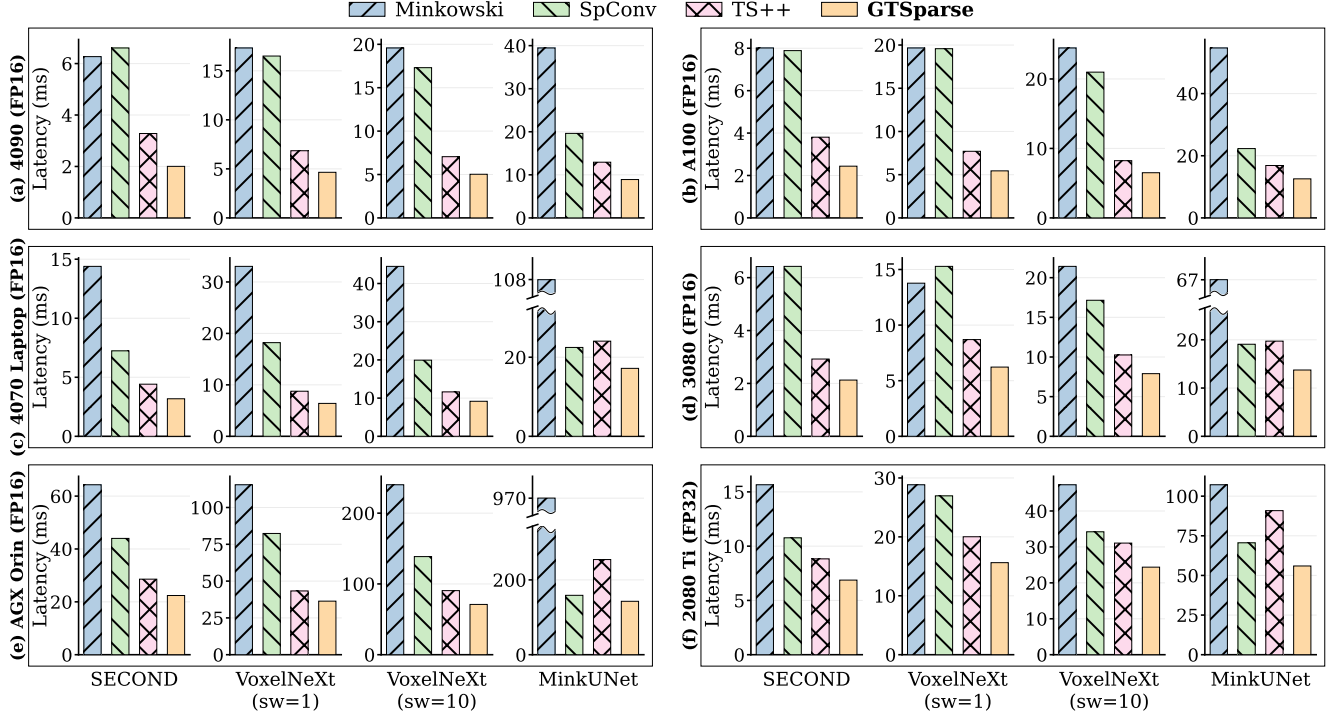


Figure 4. End-to-end inference latency across GPU platforms (FP16 unless noted). Each panel shows one GPU with four workload configurations, each plotted with an independent y-axis (latency in ms) due to differing latency ranges. Within each workload, four bars compare Minkowski, SpConv, TS++, and GTSparse. Detection workloads report sparse convolution latency; segmentation reports full pipeline latency. Lower is better.

Table 2. Template assignment distribution (% of output voxels per family). The template set is constructed from KITTI/SECOND but generalizes across datasets.

Workload	Template Family (%)				Avg Width
	center	skip(2,3)	skip(1,3)	full27	
SECOND	3.4	33.3	42.8	20.5	17.0
VoxelNeXt (sw=1)	5.1	30.6	47.6	16.7	16.7
VoxelNeXt (sw=10)	2.3	34.4	46.2	17.1	16.9
MinkUNet42	5.8	32.0	30.7	31.5	17.6

6.3 Microbenchmark

Template coverage and transferability. Table 2 reports the fraction of output voxels assigned to each template family, aggregated across all convolution layers in each workload. The template set is constructed once from SECOND / KITTI geometry and applied without modification to nuScenes and SemanticKITTI. Across all workloads, the majority of output rows are classified into narrow templates (skip(2,3) and skip(1,3)), with average effective width ranging from 16.7 to 17.6 out of a maximum 27. This confirms two properties: (1) the geometric regularity exploited by templates is not

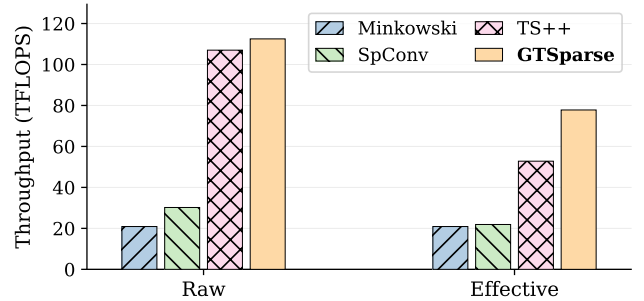


Figure 5. Raw and effective computation throughput on VoxelNeXt/nuScenes (sw=1, FP16, RTX 4090). TS++ and GTSparse achieve comparable raw throughput (>100 TFLOPS), but TS++ converts only 49.3% into useful work. SpConv’s sort+skip reduces issued computation but its preprocessing overhead limits raw throughput to 30.2 TFLOPS. GTSparse achieves 69.2% proportionality, comparable to SpConv’s 72.5% but at 3.7× the raw throughput.

dataset-specific but reflects the intrinsic spatial structure of LiDAR point clouds, and (2) most output rows require far fewer than 27 active offsets, validating the premise that full-width execution wastes substantial computation. The

Table 3. Time breakdown (ms) on RTX 4090, FP16. Builder time measures runtime construction (coordinate hashing, indice computation, or template classification); kernel time measures sparse convolution execution.

Workload	System	Builder	Kernel	Total
VoxelNeXt (sw=1)	Minkowski	2.4	17.9	20.2
	SpConv	15.2	16.6	32.4
	TS++	8.6	10.0	19.4
	GTSpase	5.9	6.4	12.3
VoxelNeXt (sw=10)	Minkowski	5.6	20.7	33.2
	SpConv	13.7	17.7	36.3
	TS++	8.7	14.2	22.6
	GTSpase	8.2	8.4	16.6

Table 4. Peak GPU memory usage (MB) on RTX 4090, FP16. GTSpase’s worst-case pre-allocation incurs modest additional memory compared to baselines.

Workload	Minkowski	SpConv	TS++	GTSpase
SECOND	64.3	54.3	84.6	115.0
VoxelNeXt (sw=1)	60.7	34.5	56.1	90.7
VoxelNeXt (sw=10)	145.3	89.2	148.2	234.2
MinkUNet42	392.7	275.4	234.8	421.2

resulting computation reduction is 34.8–38.1% compared to full-width fused-offset execution.

Effective computation throughput. Figure 5 compares both raw throughput (total issued FLOPs / time) and effective throughput T_{eff} (Eq. 3) across systems on VoxelNeXt/nuScenes (sw=1). TS++ achieves 107.0 TFLOPS raw throughput by executing a single large GEMM that saturates the GPU, but only 49.3% of that computation is useful (52.8 TFLOPS effective). SpConv achieves higher proportionality (72.5%) through sort+skip, but its sorting preprocessing overhead limits raw throughput to only 30.2 TFLOPS, resulting in 21.9 TFLOPS effective. GTSpase achieves 112.5 TFLOPS raw (comparable to TS++) while converting 69.2% into useful work (77.8 TFLOPS effective). The proportionality is comparable to SpConv (69.2% vs. 72.5%), but GTSpase operates at 3.7× the raw throughput because template classification is integrated into the builder rather than requiring a separate sort pass on the critical path. Compared to TS++, the proportionality gap (69.2% vs. 49.3%) directly translates into the end-to-end speedup.

Time breakdown. Table 3 decomposes total latency into builder time (runtime construction: hash table build, template classification, row grouping) and kernel time (sparse convolution execution). For SubM convolution, the builder executes once and its output is reused across all layers sharing the same kernel size, amortizing the construction cost.

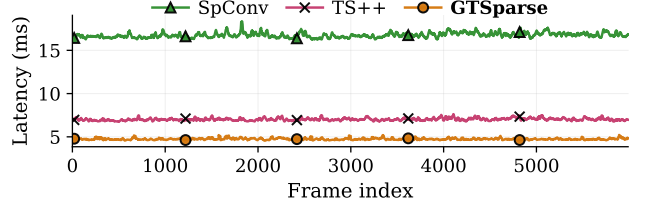


Figure 6. Per-frame latency over a continuous sequence (VoxelNeXt/nuScenes, sw=1, FP16, RTX 4090). All systems exhibit correlated frame-to-frame variation driven by input density, but GTSpase remains consistently below baselines throughout the sequence.

On VoxelNeXt (sw=1), GTSpase’s builder adds 5.9 ms (48% of total), while the kernel time is 6.4 ms compared to 10.0 ms for TS++. SpConv’s sort-based builder is the most expensive (15.2 ms), consuming 47% of its total latency. The builder overhead is modest relative to the kernel savings because template classification and grouping operate on N_{out} scalar comparisons (bitmask tests), whereas the kernel reduction applies to $N_{\text{out}} \times K \times C$ multiply-accumulate operations. As density increases (sw=10), builder cost grows moderately for all systems while kernel time increases proportionally, maintaining GTSpase’s advantage.

Memory overhead. Table 4 reports peak GPU memory usage. GTSpase pre-allocates each template’s payload buffer at worst-case size $\hat{N} \times |T_i|$ integers (where $\hat{N} = \lceil |\mathcal{A}_{\text{out}}| / B_M \rceil \cdot B_M$), totaling $\hat{N} \times \sum_i |T_i|$ integers across all templates (§5). This avoids dynamic allocation during inference but incurs 30–186 MB additional memory compared to TS++ across workloads. The overhead is acceptable on all evaluated platforms: even on AGX Orin (32 GB unified memory shared with CPU), the additional allocation represents less than 0.6% of available memory. The pre-allocation strategy trades a bounded memory increase for deterministic builder execution without cudaMalloc calls on the inference path.

Per-frame latency stability. Figure 6 plots per-frame latency over a continuous evaluation sequence. All systems exhibit frame-to-frame variation that tracks the input point cloud density (frames with more active voxels take longer across all systems). GTSpase’s latency curve remains consistently below the baselines throughout the sequence, with the speedup ratio stable across frames regardless of density variation. This confirms that template-driven execution does not introduce additional latency jitter: the per-frame variation is workload-intrinsic (shared by all systems), and GTSpase’s speedup is robust across the natural density distribution of real driving sequences.

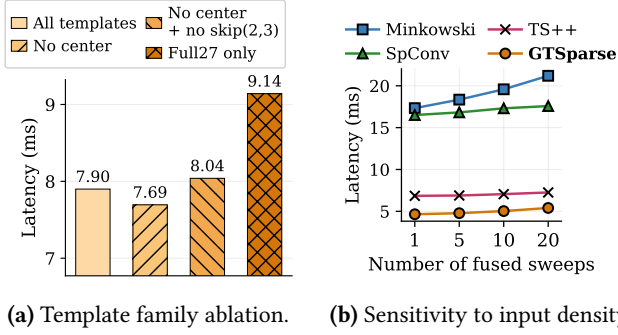


Figure 7. (a) Effect of progressively removing template families (VoxelNeXt/nuScenes, sw=10, FP16, RTX 3080). (b) Latency vs. sweep count (VoxelNeXt/nuScenes, FP16, RTX 4090; Minkowski in FP32).

6.4 Ablation and Sensitivity

Template family ablation. As shown in Figure 7a, we progressively remove template families in order of increasing width: first center (width 1), then skip(2, 3) (width 10), then skip(1, 3) (width 19), leaving only the full27 fallback. Removing center slightly improves latency (7.90 ms $\rightarrow$ 7.70 ms) because eliminating its small group reduces tile padding overhead. Further removing skip(2, 3) increases latency to 8.04 ms (+4.5%), and removing skip(1, 3) produces the largest single degradation (8.04 ms $\rightarrow$ 9.14 ms, +13.7%), as this family covers the most output rows. With only full27 remaining, latency approaches TS++ (9.14 ms vs. 10.26 ms), confirming that template-driven width reduction is the primary source of speedup over fused-offset baselines.

Sensitivity to input density. Figure 7b varies the number of fused LiDAR sweeps (1, 5, 10, 20) on VoxelNeXt/nuScenes (FP16, RTX 4090) to sweep input density from $\sim 0.02\%$ to $\sim 0.20\%$ active ratio. All systems’ latencies increase with density, but at different rates: GTSparse grows from 4.65 ms to 5.40 ms (+16%), while TS++ grows from 6.85 ms to 7.26 ms (+6%). The speedup over TS++ correspondingly narrows from $1.47\times$ (sw=1) to $1.34\times$ (sw=20), because denser inputs fill more offsets per output row, shifting template classification toward wider families. The relationship is monotonic but sub-linear: the relative speed up shows that even at 20-sweep density, the average effective width remains well below 27. This confirms that template-driven execution remains beneficial across the density range encountered in practice, with the largest gains at the low densities typical of single-sweep autonomous driving workloads.

6.5 Discussion

Scope and limitations. The evaluation focuses on batch=1 inference, the dominant deployment mode for latency-sensitive 3D perception. Batched execution would require per-sample template assignments and inter-sample load balancing; the

geometric template abstraction itself is batch-agnostic (each sample’s classification is independent), but the builder and tile stream layout would need extension. GTSparse’s worst-case pre-allocation trades memory for builder simplicity (Table 4); this overhead is negligible on all evaluated platforms including AGX Orin (less than 0.6% of available memory).

Template set optimality. The current template set (8 templates in 4 families) is constructed from geometric reasoning over the $3\times 3\times 3$ kernel and validated by the ablation in Figure 7a, which confirms that each family contributes measurable gains. However, the number of templates, their offset compositions, and the assignment policy are all degrees of freedom that could be jointly optimized. The primary contribution of this paper is the template-driven execution runtime itself, which decouples the template set from the execution machinery; exploring optimal template selection within this framework is a natural direction for future work.

Data flow optimization. To isolate the contribution of the execution model, this evaluation does not incorporate intra-template sort-and-skip. The end-to-end results suggest that sorting-based computation skipping becomes valuable under heavier workloads where arithmetic cost dominates preprocessing overhead (e.g., SpConv outperforming TS++ on AGX Orin MinkUNet). Within the template-driven runtime, intra-template sorting is a natural extension whose problem space is strictly smaller than SpConv’s global sort: sorting operates independently within each template group rather than over all output rows, and each group sorts over at most the template’s offset count (1, 10, or 19) rather than the full 27. The smaller sort inputs and narrower bitmasks should lower the sorting overhead relative to the computation it eliminates. Integrating this optimization is a natural next step for denser workloads and embedded platforms.

7 Conclusion

We presented GTSparse, a geometric-template-driven sparse convolution runtime that resolves the fundamental tension between no wasted computation and high throughput in GPU sparse convolution. A compact, fixed, input-independent template set simultaneously approaches no wasted computation through narrower offset widths and preserves high throughput through unified dense GEMMs within each template group, delivering consistent speedups over all evaluated baselines while producing numerically identical results. As LiDAR perception systems adopt higher-resolution sensors and denser temporal fusion, sparse convolution latency increasingly bounds the end-to-end perception pipeline. By reducing this latency across the full hardware deployment spectrum, GTSparse improves the decision horizon and responsiveness of latency-critical 3D perception applications, including autonomous driving, robotics, and AR/VR.

References

- [1] Jens Behley, Martin Garbade, Andres Milioto, Jan Quenzel, Sven Behnke, Cyrill Stachniss, and Jurgen Gall. 2019. SemanticKITTI: A Dataset for Semantic Scene Understanding of LiDAR Sequences. In *ICCV*.
- [2] Holger Caesar, Varun Bankiti, Alex H. Lang, Sourabh Vora, Venice Erin Liong, Qiang Xu, Anush Krishnan, Yu Pan, Giancarlo Baldan, and Oscar Beijbom. 2020. nuScenes: A Multimodal Dataset for Autonomous Driving. In *CVPR*. <https://doi.org/10.1109/CVPR42600.2020.01164>
- [3] Yukang Chen, Jianhui Liu, Xiangyu Zhang, Xiaojuan Qi, and Jiaya Jia. 2023. VoxelNeXt: Fully Sparse VoxelNet for 3D Object Detection and Tracking. In *CVPR*.
- [4] Christopher Choy, JunYoung Gwak, and Silvio Savarese. 2019. 4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks. In *CVPR*.
- [5] Angela Dai, Matthias Nießner, Michael Zollhöfer, Shahram Izadi, and Christian Theobalt. 2017. BundleFusion: Real-Time Globally Consistent 3D Reconstruction Using On-the-Fly Surface Reintegration. *ACM Trans. Graph.* 36, 3 (2017). <https://doi.org/10.1145/3054739>
- [6] Lue Fan, Feng Wang, Ziqi Pang, Lei Zhu, Zhicheng Su, Fang Chen, Naiyan Wang, and Zhaoxiang Zhang. 2022. Embracing Single Stride 3D Object Detector with Sparse Transformer. In *CVPR*.
- [7] Andreas Geiger, Philip Lenz, and Raquel Urtasun. 2012. Are we ready for autonomous driving? The KITTI vision benchmark suite. In *CVPR*. <https://doi.org/10.1109/CVPR.2012.6248074>
- [8] Shahram Izadi, David Kim, Otmar Hilliges, David Molyneaux, Richard A. Newcombe, Pushmeet Kohli, Jamie Shotton, Steve Hodges, Dustin Freeman, Andrew J. Davison, and Andrew W. Fitzgibbon. 2011. KinectFusion: Real-time 3D Reconstruction and Interaction Using a Moving Depth Camera. In *UIST*. <https://doi.org/10.1145/2047196.2047270>
- [9] Alex H. Lang, Sourabh Vora, Holger Caesar, Lubing Zhou, Jiong Yang, and Oscar Beijbom. 2019. PointPillars: Fast Encoders for Object Detection From Point Clouds. In *CVPR*. <https://doi.org/10.1109/CVPR.2019.01298>
- [10] Yujun Lin, Zhekai Wang, Yingyan Ding, and Song Han. 2021. PointAcc: Efficient Point Cloud Accelerator. In *MICRO*.
- [11] Zhijian Liu, Haotian Tang, Alexander Amini, Xinyu Yang, Huizi Mao, Daniela L. Rus, and Song Han. 2023. BEVFusion: Multi-Task Multi-Sensor Fusion with Unified Bird’s-Eye View Representation. In *ICRA*. <https://doi.org/10.1109/ICRA48891.2023.10160968>
- [12] Wenjie Luo, Bin Yang, and Raquel Urtasun. 2018. Fast and Furious: Real Time End-to-End 3D Detection, Tracking and Motion Forecasting With a Single Convolutional Net. In *2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018*. Computer Vision Foundation / IEEE Computer Society, 3569–3577. <https://doi.org/10.1109/CVPR.2018.00376>
- [13] Tixiao Shan and Brendan J. Englot. 2018. LeGO-LOAM: Lightweight and Ground-Optimized Lidar Odometry and Mapping on Variable Terrain. In *IROS*. <https://doi.org/10.1109/IROS.2018.8594299>
- [14] Tixiao Shan, Brendan J. Englot, Drew Meyers, Wei Wang, Carlo Ratti, and Daniela Rus. 2020. LIO-SAM: Tightly-coupled Lidar Inertial Odometry via Smoothing and Mapping. In *IROS*. <https://doi.org/10.1109/IROS45743.2020.9341176>
- [15] Pei Sun, Henrik Kretschmar, Xerxes Dotiwalla, Aurelien Chouard, Vijaysai Patnaik, Paul Tsui, James Guo, Yin Zhou, Yuning Chai, Benjamin Caine, Vijay Vasudevan, Wei Han, Jiquan Ngiam, Hang Zhao, Aleksei Timofeev, Scott Ettinger, Maxim Krivokon, Amy Gao, Aditya Joshi, Yu Zhang, Jonathon Shlens, Zhifeng Chen, and Dragomir Anguelov. 2020. Scalability in Perception for Autonomous Driving: Waymo Open Dataset. In *CVPR*. <https://doi.org/10.1109/CVPR42600.2020.00252>
- [16] Haotian Tang, Zhijian Liu, Shengyu Zhao, Yujun Lin, Ji Lin, Hanrui Wang, and Song Han. 2022. TorchSparse: Efficient Point Cloud Inference Engine. In *MLSys*.
- [17] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. PCEngine: Efficient Point Cloud Inference Engine. In *MLSys*.
- [18] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. TorchSparse++: Efficient Training and Inference Framework for Sparse Convolution on GPUs. In *MICRO*.
- [19] Thomas Whelan, Stefan Leutenegger, Renato F. Salas-Moreno, Ben Glocker, and Andrew J. Davison. 2015. ElasticFusion: Dense SLAM Without A Pose Graph. In *Robotics: Science and Systems*. <https://doi.org/10.15607/RSS.2015.XI.001>
- [20] Benjamin Wilson, William Qi, Tanmay Agarwal, John Lambert, Jagjeet Singh, Siddhesh Khandelwal, Bowen Pan, Ratnesh Kumar, Andrew Hartnett, Jhony Kaesemodel Pontes, Deva Ramanan, Peter Carr, and James Hays. 2021. Argoverse 2: Next Generation Datasets for Self-Driving Perception and Forecasting. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*, Joaquin Vanschoren and Sai-Kit Yeung (Eds.). <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/4734ba6f3de83d861c3176a6273cac6d-Abstract-round2.html>
- [21] Yan Yan, Yuxing Mao, and Bo Li. 2018. SECOND: Sparsely Embedded Convolutional Detection. *Sensors*.
- [22] Zetong Yang, Yin Zhou, Zhifeng Chen, and Jiquan Ngiam. 2021. 3D-MAN: 3D Multi-Frame Attention Network for Object Detection. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2021, virtual, June 19-25, 2021*. Computer Vision Foundation / IEEE, 1863–1872. <https://doi.org/10.1109/CVPR46437.2021.00190>
- [23] Tianwei Yin, Xingyi Zhou, and Philipp Krähenbühl. 2021. Center-based 3D Object Detection and Tracking. In *CVPR*.
- [24] Ji Zhang and Sanjiv Singh. 2014. LOAM: Lidar Odometry and Mapping in Real-time. In *Robotics: Science and Systems*. <https://doi.org/10.15607/RSS.2014.X.007>